Bocconi

# COMPUTER SCIENCE

code 30424 a.y. 2024-2025

## Lesson 16

Functions and exceptions handling

Università Bocconi
MILANO

# Objectives of the lesson

- Learn how to create custom functions
- Learn how to handle errors

# Functions in Python

- In Python there are many functions, called **built-in functions**, that are immediately usable (e.g. print, input, str, int and float)
- Many other functions are available in Python's **standard library** modules, such as **math** and **random**, or in additional third-party libraries
- In addition to these functions, it is also possible to create **custom functions** to perform tasks and operations based on our needs

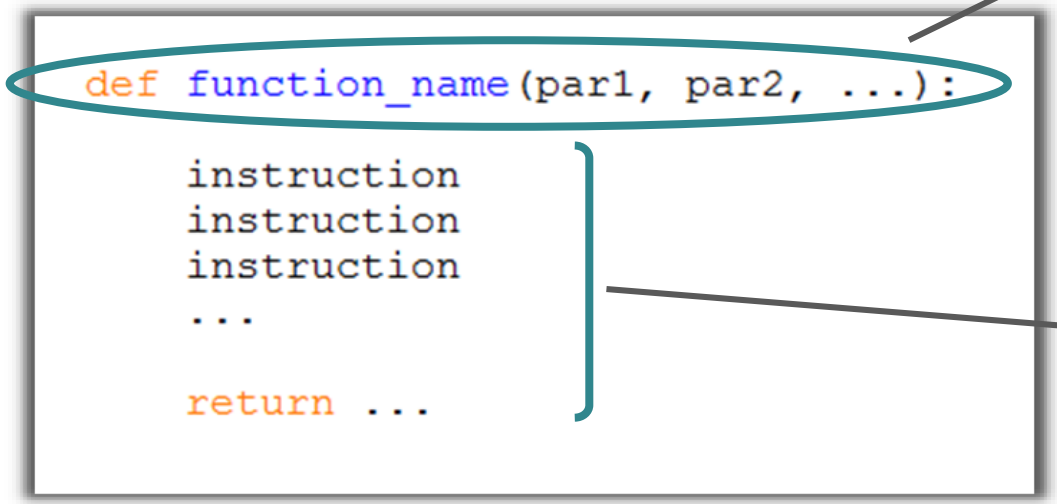
# Functions

- A function contains a set of instructions, grouped under a given name, used to perform a specific task within a program
- Functions can perform tasks of any type: they can execute a calculation, perform an action, create an object, update values, etc.
- In addition to performing calculations or actions, functions are important because they facilitate the creation of a **modular program**

# Creating custom functions

A function is made up of two parts:

- header
- body




The header contains:

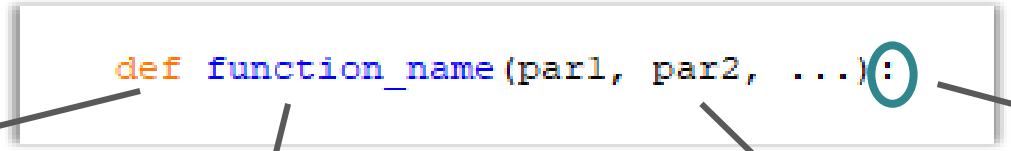
- the keyword def
- the name of the function
- the list of parameters between parentheses
- a colon

The body:

- contains all the instructions executed by the function
- can end with the return statement

# The header of a function

Each element of the header has a specific task:




The keyword def is needed to start *defining* a function

The header always ends with the symbol « : »

The name uniquely identifies the function in the program.

To assign the function name, we must follow some rules:

- the first character must be a letter or an underscore
- keywords or spaces cannot be used
- after the first letter, we can use letters (uppercase or lowercase), numbers or underscores
- Python is case sensitive, i.e. it distinguishes between uppercase and lowercase characters

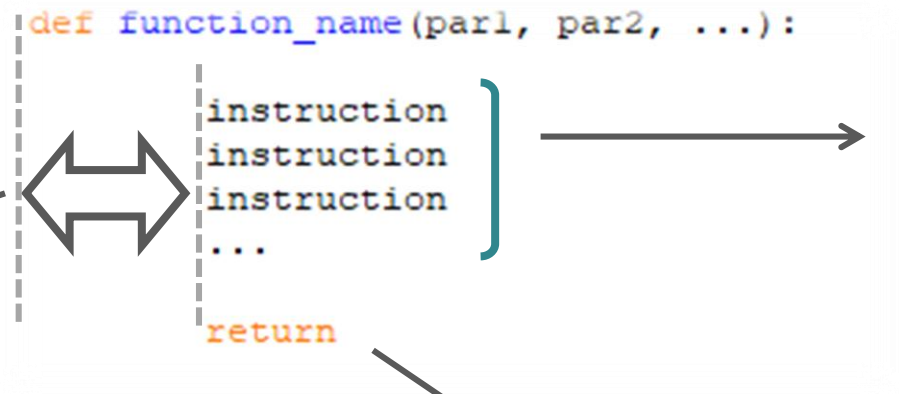
Parameters specify which arguments will be passed to the function:

- names are separated by commas
- parameters can be mandatory or optional

# The body of a function

The body of a function contains a list of instructions (called block):

- all the instructions and the return statement must be indented
- it ends with the return statement when the function must return a value to the calling instruction; in all other cases, it ends without the return statement




Each block can contain any type of instruction: calculations, conditional statements, call of other functions, etc.

All the lines of code contained in the body of the function must be indented

The return statement is necessary if the function must return a result to the calling instruction

# Calling a function

- Defining a function, Python understands which task must be performed by the function, but it does not execute it
- Each time we want to execute a function it is necessary to **call it** by using the syntax:

```
function_name()
```

- If parameters have been specified in the definition of a function, their value must be written in parentheses
- A function can be called in Python's shell, by a program or by another function

# Examples of definition and call of a function

```
def hello():
  print('Hello my friend, is everything ok?')
  print('Hope so!')
```

```
>>> hello()
Hello my friend, is everything ok?
Hope so!
```

```
def pleased():
    name = input("What's your name? ")
    print('Pleased to meet you, ' + name + '!')
```

```
>>> pleased()
What's your name? Martin
Pleased to meet you, Martin!
```

```
def square(x):
    print('The square of', x, 'is:', x*x)
```

```
>>> square(5)
The square of 5 is: 25
```

# Function arguments

- We often need to pass some inputs to a function: these inputs, called arguments, are used by the function to perform its task and they must be defined through a list of parameters in the function's definition
- Parameters:
  - explain to Python that data must (or can) be passed to the function when it is called
  - allow to specify how the data should be used to perform the function's tasks
- Parameters are specified in the parentheses after the name of the function, separated by commas:

```
def division(num1, num2):
    print('The result of the division is:', num1/num2)
```

# Mandatory and optional parameters

- Parameters can be mandatory or optional: when defining a function, it is always necessary to specify all the mandatory parameters first, then the optional ones
- For each optional parameter, the corresponding default value must be defined, i.e. the value applied automatically by Python if the optional argument is not specified in the function call
- In the function definition, optional parameters are specified in the following way:

*parameter=default_ value*

- The general syntax to define a function with parameters is therefore the following:

```
def function(par1, par2, par3=value, par4=value):
```

# Calling a function with parameters

- If mandatory parameters are specified when defining a function, a corresponding number of arguments must be provided when the function is called
- When a function with parameters is called, it is necessary to specify the arguments by position (in the same order in which the parameters are defined) or by using the parameters name (keyword arguments)
- In the function call it is possible to use both arguments by position and keyword arguments, but in this case it is mandatory to write first the arguments by position, then the keyword arguments
- It is possible to pass a variable as an argument to a function

# Example of definition and call of functions with parameters

```
def calc(par1, par2, par3=10):
    total = (par1 + par2) / par3
    return total
```

```
>>> calc()
Traceback (most recent call last):
  File "<pyshell#7>", line 1, in <module>
    calc()
TypeError: calc() missing 2 required positional arguments: 'par1' and 'par2'
```

```
>>> calc(15,5)
2.0
```

```
>>> calc(20,80,5)
20.0
```

```
>>> calc(par2=50,par3=20,par1=30)
4.0
```

```
>>> calc(10, par2=5)
1.5
```

```
>>> calc(10, par3=3, par2=5)
5.0
```

```
>>> calc(par2=3, 5)
SyntaxError: positional argument follows keyword argument
```

# Productive and void functions

- A **productive** function is a set of instructions that performs a specific task and **returns**, when it ends, **a value to the instruction that called it**
- A **void** function performs a specific task and **does not return any value to the instruction that called it**
- Productive functions always end with the return statement at the end of their body: the instructions after the return statement are not executed
- The difference between void and productive functions therefore consists in:
  - the return statement (to be applied only to productive functions)
  - the ability to return a value to the instruction that called them, so that the value can be stored in a variable or reused

# Productive and void functions compared

| Productive function | Void function |
| --- | --- |
| `def square(x):`<br>`    return x*x` | `def square(x):`<br>`    print('The square of', x, 'is:', x*x)` |
| `>>> y = square(10)`<br>`>>> y`<br>`100` | `>>> y = square(10)`<br>`The square of 10 is: 100`<br>`>>> print(y)`<br>`None` |
| The function returns a value and it is stored in the variable **y**, so that it can be used in other instructions | The function shows a text string on screen and returns no value, that's why variable **y** is empty. Therefore, the function's output cannot be used in other instructions |

# Local and global variables

- A **global** variable can be reached by any instruction in a program
- A **local** variable can be reached only within its scope, that is within the part of the program in which it was defined, such as a function
- The variables created within a function are local variables, therefore they cannot be reached outside of the function

```
def calc(x, y):
    temp1 = (x + y) / y**2
    temp2 = x**2 - y**2
    return temp1 - temp2
```

```
>>> print(calc(1,1))
2.0
>>> print(temp1)
Traceback (most recent call last):
  File "<pyshell#14>", line 1, in <module>
    print(temp1)
NameError: name 'temp1' is not defined
```

# The docstring

- When creating a function, it is possible to insert a few lines of explanation by defining a docstring
- The docstring is applied in the function definition and consists of a text string, enclosed by triple quotes, written on one or more lines immediately after the header
- The docstring content is visible when reading the code, when using the help function or in the *call tip*

```
def multiply(num1, num2, num3):
    ''' The function multiplies the three arguments and returns the result.\n \
        Arguments can be integer or decimal numbers'''
    return num1 * num2 * num3


>>> multiply(
```

```
(num1, num2, num3)
The function multiplies the three arguments and returns the result.
Arguments can be integer or decimal numbers
```

# Functions with loops and conditional statements

```
def MyProduct(x,y):
    total = 0
    for i in range(y):
        total = total + x
    return total
```

```
def odd_even(num):
  count_odd = 0
  count_even = 0
  for n in range(1,num+1):
    if n % 2 == 0:
      count_even = count_even + 1
    else:
      count_odd = count_odd + 1
  print("Number of even numbers :",count_even)
  print("Number of odd numbers :",count_odd)
  return count_even
```

```
def discount(quant, price):
  if quant > 100:
    return quant * price * (1 - 0.4)
  elif quant > 50:
    return quant * price * (1 - 0.2)
  else:
    return quant * price
```

# Exercise 1

Create the Python file **Exercise 1.py** containing a function, called **sum_range**, which must:

- have an integer as a mandatory parameter
- calculate, using a loop controlled by a counter, the sum of all the integer numbers between 1 and the number received as mandatory argument (both inclusive)
- return the sum just calculated

# Errors in Python

Errors can be divided into three categories:

- Syntax errors
- Runtime errors
- Semantic errors

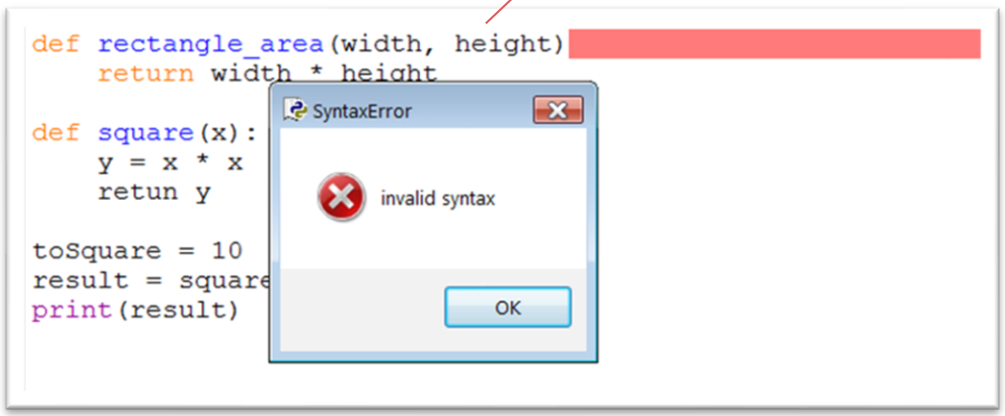
# Syntax errors

- Syntax errors take place when there is an error in how the code is written
- Python shows an error message (traceback message) in the shell and shows where the error occurs, but it does not specify how to correct it
- The error message always starts with SyntaxError

In the shell (after Enter)

```
>>> print("Result:"; 6+4)
    SyntaxError: invalid syntax
>>>
```

In the editor (after Run)



# Runtime errors

- Runtime errors occur when there is an error in the code, even if the syntax is correct
- Python shows an error message (in the shell only) containing the line of code that generates the error and the error type

```
>>> print(result)
Traceback (most recent call last):
  File "<pyshell#0>", line 1, in <module>
    print(result)
NameError: name 'result' is not defined
>>>
```

```
>>> print(1/0)
Traceback (most recent call last):
  File "<pyshell#1>", line 1, in <module>
    print(1/0)
ZeroDivisionError: division by zero
>>>
```

# Semantic errors

- Semantic errors occur when the program is executed without producing error messages, but the results are not correct (they are inconsistent or unexpected)
- They are caused by a wrong code design (they are also called logic errors)
- They are tricky to find and require a step by step analysis of the code or the use of more sophisticated debugging tools **(Debugger)**

# Exceptions

- An exception in Python is an error that occurs during the execution of a program, disrupting the normal flow of instructions
- When an exception occurs, Python raises a runtime error, which – if not handled properly – will cause the program to terminate
- If we expect an exception to occur, we can write some instructions to handle it (*exception handling*), preventing the program from abruptly ending

# Exceptions handling

- Handling errors means instructing the program on what to do if an exception occur
- To handle errors Python provides the statements try and except
- It is possible to specify a unique except procedure to handle all possible errors or different except procedures to handle specific types of error

# Example

Let's create a program that performs a division between two integer numbers entered by the user. The program must prevent any kind of error, i.e.:

- text strings or decimal numbers entered instead of integer numbers (ValueError)
- a zero value entered as denominator (ZeroDivisionError)
- any other error

```
try:
    a = int(input('Enter a number: '))
    b = int(input('Enter another number: '))
    print(a/b)

except ValueError:
    print('\nEnter integer numbers only!')
except ZeroDivisionError:
    print("\nOne of the two values is 0! \n \
    Please try again from the beginning with another number")
except:
    print('\nSomething's wrong: try again with other numbers!')
```

# Debugging

- Debugging consists in searching and fixing code errors
- Debugging takes place re-reading the code, searching for errors, modifying the code and re-executing the program
- There are specific tools for debugging (e.g. the Debugger)

# Exercise 2

In a new Python file, named **Exercise 2.py**, create a function that:

- receives an integer (different than 0) as mandatory argument
- counts how many integers between 1 and the number passed as argument to the function (both inclusive) are divisible by 3 and by 7 but not by 5
- returns the result of the previous count

Then, create a copy of the function and modify it in order to handle the cases in which the argument passed to the function is a text string or a floating-point number: in these cases, the function should return the string 'Wrong argument type'

# Tools to learn with today's lecture

- Creating custom functions
- Handling errors

# Book references

**Understanding Python**:

Chapters 7, 10.4, 10.5, 10.6

**Python workbook**:

Exercises of Units 5, 6